ChatGPT

HTML UI Service for AWS Deployment (Project Brief)

Project Overview

This project involves creating a **web-based HTML UI service** that integrates with the existing build orchestrator to facilitate automated AWS environment setup. The UI service will act as an interface for deploying and managing infrastructure on AWS. It leverages an underlying **OSBot-AWS** library (from the OWASP Security Bot toolkit) to simplify AWS operations and will expose those operations as easy-to-use functions. By providing a user-friendly front-end and a RESTful API for AWS tasks, the service aims to enable on-demand provisioning of the entire application stack in AWS with minimal manual intervention.

Objectives and Scope

- **On-Demand Environment Deployment:** Allow users or automated processes to spin up and tear down the **entire application infrastructure** from scratch on AWS, in any specified region or environment, through a simple UI or API call. This includes creating all necessary cloud resources (compute, storage, networking, etc.) on demand.
- **Integration with Build Orchestrator:** The UI service will be **tightly integrated with the existing build service**, which acts as the orchestration engine. The build orchestrator will call the UI service's API endpoints to perform each infrastructure action in the correct sequence. All orchestration logic (the "when and what to execute" ordering of steps) remains in the build service, while the UI service focuses on performing individual AWS operations.
- **Expose AWS Primitives as API Calls:** Rather than define complex workflows internally, the UI service's scope is to provide **primitive operations** for AWS. Each key AWS action will be a dedicated function or endpoint (for example: "create S3 bucket", "check S3 bucket exists", "create AMI", "launch EC2 from AMI", "create Lambda function", etc.). These granular building blocks can be invoked one by one by the orchestrator. This approach ensures flexibility in how AWS resources are composed and sequenced without hardcoding orchestration into the UI service.
- **Secure Credential Handling:** The system will use a **secure pass-through for credentials** following a proven "public key encryption" approach (inspired by the earlier GitHub integration). The build orchestrator itself will not store or directly handle sensitive AWS credentials in plain text. Instead, the UI service will provide its public key to the orchestrator, which will use it to encrypt the required AWS access keys/tokens. The encrypted credentials are then passed to the UI service for decryption and usage. This ensures that AWS keys are only usable within the secure context of the UI service when performing the actions, significantly reducing exposure of secrets.
- **Scope Limitations:** This project focuses on the AWS UI service component **only**. It will implement AWS resource management functionality and a basic interface for triggering them. Broader concerns like multi-cloud support, complex orchestration logic, or long-term state management (e.g. tracking infrastructure state as Terraform does) are **out of scope**. The UI service assumes AWS is the target platform and that the existing build orchestrator will handle any higher-level logic such as conditional flows or waiting between steps.

Key Features and Functionality

- **User Interface:** A lightweight web-based UI will be provided for human users. This HTML interface may allow entering configuration parameters (like selecting AWS region, environment name, or providing keys in an encrypted form) and then triggering deployment actions. The goal is to make it easy for a developer or operator to kick off deployments or resource creation without needing to use the AWS console or CLI directly.
- **RESTful API Endpoints:** Every AWS action will be exposed as a REST API call, enabling automation and integration. For example, the service will have endpoints such as:
- `/create-s3-bucket` – to create an S3 bucket (with necessary parameters like bucket name, region, etc.).
- `/check-s3-bucket` – to verify if a given S3 bucket exists or is accessible.
- `/create-ami` – to create an Amazon Machine Image (AMI) from a specified source (e.g. an EC2 snapshot or instance).
- `/launch-ami-instance` – to launch an EC2 instance from a given AMI, possibly with user data scripts to configure it.
- `/create-target-group` – to set up a target group for load balancing (for the new instances or services).
- `/create-auto-scaling-group` – to create an Auto Scaling Group and attach it to the target group, ensuring the application can scale out/in.
- `/create-lambda-function` – to deploy an AWS Lambda function, including setting its code, configuration, and enabling a Function URL if needed for web access.
- (And similarly, endpoints for other necessary resources like creating SQS queues, DynamoDB tables, IAM roles, etc., as required by the application's infrastructure.)
- **Primitive Operations with Verification:** For each create action, there will likely be a corresponding **verification or status check** action. For example, after calling create, the build orchestrator might call a "check" endpoint to ensure the resource was created successfully (e.g., check if an S3 bucket is now present, or check if a Lambda's endpoint is responding). These verification endpoints let the orchestrator confirm each step before moving on, enabling robust error handling and conditional logic if something fails. The UI service will implement these checks using AWS SDK calls (via OSBot-AWS) to inspect the current state of resources.
- **Leverages OSBot-AWS Library:** Under the hood, the service will utilize the OSBot-AWS Python library, which provides convenient, high-level wrappers over AWS Boto3 SDK calls. This means each endpoint in the UI service can call a simple OSBot function to perform the desired action on AWS. Using this library accelerates development and ensures consistency, as it abstracts away low-level AWS complexities in favor of more intuitive operations. (For instance, a single OSBot-AWS call might handle creating an S3 bucket with proper permissions, or deploying a Lambda with the right configuration, instead of writing extensive Boto3 code each time.)
- **Full Application Deployment Capability:** By combining the above primitives, the system will be capable of **bringing up a full application stack**. For example, a typical deployment workflow that the build orchestrator might execute through the UI service could be:
- **Infrastructure Setup:** Create a new network stack (VPC, subnets) if needed, or use existing defaults. *(If networking setup is handled elsewhere, this step might be skipped or minimal.)*
- **Storage and Data Layer:** Create required S3 buckets (for assets, logs, backups, etc.) and any database instances or tables (if part of the app, though database may be separate).
- **Compute Layer:** Create a new AMI from the application's latest build or base image. Then launch EC2 instances from this AMI inside an Auto Scaling Group for the application servers.
- **Networking & Load Balancing:** Set up a Target Group for the application, and attach it to a load balancer (could be an ALB/ELB) that directs traffic to the EC2 instances. Register the new instances with the target group. Also configure health checks via the target group or load balancer.

- **Serverless Components:** Deploy any AWS Lambda functions that are part of the system (for example, for background jobs or API endpoints), and configure their Function URLs or API Gateway triggers as needed.
- **Post-Deployment Checks:** Verify that all components are active and healthy – e.g., ensure the EC2 instances are running and passing health checks, the load balancer is up, Lambdas are reachable, and buckets are accessible.
- **Tear-Down (if triggered):** Conversely, the service will also support destroying or removing these resources when they are no longer needed, to allow true on-demand environment lifecycle management.
- **Different Modes / Configurations:** The UI service will be designed to support multiple **deployment modes**, adapting to various scenarios:
- **Fresh Deployment vs. Update:** It can distinguish between bringing up a brand new environment versus updating an existing one. (For a fresh deployment, it creates everything from scratch; for updates, it might skip creation of resources that already exist or use a different endpoint to update configurations.)
- **Secure Mode Operation:** As noted, in secure mode the service runs with no persistent credentials; it receives encrypted credentials at runtime. There might also be a mode for development or testing where the service could use pre-configured credentials (for example, if running in an environment where AWS keys are available via IAM roles or environment variables, the service could use those directly without the public-key exchange).
- **Orchestration Mode:** While the UI service itself does not orchestrate, it could potentially offer a **combined action** for convenience (like a single "deploy environment" call for testing that internally calls multiple primitives in order). However, in the primary mode, it stays as a collection of individual calls so that the external build system fully controls the sequence. This design makes the system flexible and easy to integrate into complex pipelines or even to be driven by an AI agent or script.
- **Logging and Feedback:** Every action triggered via the UI service will produce logs or feedback that can be observed by the user or the orchestrator. The UI will likely display status messages or results of each operation (success/failure and details). If an operation fails (e.g., bucket creation fails due to name conflict), the service will report that back so that the orchestrator can halt the sequence or take corrective action. This feedback mechanism is crucial for troubleshooting and ensuring the automation is reliable. In the UI itself, this may be shown as a real-time log or a summary of what was done for a deployment session.

Integration and Architecture

- **Build Orchestrator Connection:** The existing build service acts as the "brain" of deployment, and the new UI service is one of its "hands." The integration will be achieved via **REST API calls** – the build orchestrator will call the appropriate endpoint on the UI service at each step of the deployment process. For instance, when a developer requests a new environment, the build service might make a sequence of calls: first to the UI service's `/create-ami`, then on success call `/launch-ami-instance`, then `/create-target-group`, and so on. The UI service will execute each call using AWS and return the result (or error) to the orchestrator.
- **Security via MitM Proxy Approach:** The credential forwarding design can be seen as a controlled man-in-the-middle pattern. The UI service will likely expose an endpoint like `/public-key` which the build orchestrator can call to retrieve the service's public encryption key. The orchestrator then uses this key to encrypt secrets (AWS Access Key ID, Secret Key, session tokens, etc.) and posts them to the secure endpoints (like as part of a deploy initiation call). The UI service, which holds the corresponding private key, decrypts the credentials locally and uses them to authenticate to AWS for the required operations. This means the build orchestrator never needs to store or see the raw credentials, and the credentials in transit are protected. This

integration pattern was tested with the GitHub PAT scenario previously and will be reused for AWS integration.

- **OSBot-AWS and AWS SDK:** Within the UI service's backend, the operations are implemented using OSBot-AWS, which in turn uses AWS's Boto3 SDK. The service acts as a thin layer between the orchestrator and AWS, translating high-level requests (like "create an auto-scaling group named X") into the specific AWS API calls needed. OSBot-AWS provides convenient functions, so the UI service can remain concise and focused on input/output handling and not the nitty-gritty of AWS APIs.
- **Stateless Service Design:** The HTML UI service will be largely stateless between requests (except for perhaps transient credential storage during an operation). It will not maintain complex state about deployments; state (such as "which resources have been created so far") is expected to be tracked by the orchestrator or by querying AWS directly. This stateless design allows the service to be scalable and simple: each request from the build orchestrator is independent, making it possible to run multiple deployment actions in parallel or recover from crashes by simply re-invoking the last failed action. Any necessary context (like an environment ID or deployment session ID) can be passed in each API call if needed, to group related actions.
- **Web UI for Visibility:** In addition to the API, the HTML interface can provide a dashboard or forms for manual control. This might show a list of environments, allow triggering a full deploy or destroy with one click, and show real-time progress logs. It's essentially a friendly layer on top of the same API the orchestrator uses. Developers could use this UI in development or testing phases to manually deploy their stack or to debug issues by trying individual steps (for example, manually invoking "create bucket" from the UI to see error messages without running the whole pipeline).

Deliverables and Next Steps

- **Functional UI Service:** The primary deliverable is a running **HTML UI service** (web application) that exposes the AWS deployment primitives as web pages and API endpoints. It should be thoroughly tested to ensure each endpoint correctly creates or checks the corresponding AWS resource.
- **API Documentation:** A clear specification of all REST endpoints, their required parameters (e.g. JSON payloads or form fields), and expected responses will be provided. This documentation will allow the build orchestrator team (and any other clients) to integrate with the service easily. It effectively serves as the contract between the UI service and the orchestrator.
- **Security Audit:** Before full integration, the credential handling flow will be reviewed to ensure encryption and decryption are implemented correctly and that no sensitive data is logged or stored inadvertently. Using AWS IAM roles or temporary credentials where possible will be considered to minimize long-lived secrets. The public/private key pair management will also be documented (how the keys are generated, rotated, etc.).
- **Integration Testing:** A series of integration tests will be performed, where the build orchestrator triggers a dummy deployment using the UI service in a sandbox AWS account. This will validate the end-to-end flow: from providing encrypted credentials, through each resource creation, to verifying the resources exist. Success criteria include being able to deploy a full mock environment and then successfully tear it down, all through automated calls.
- **Timeline:** The implementation will be iterative, likely starting with a few core primitives (for example, S3 bucket and EC2/AMI flows) and then expanding to cover all required AWS components. Early deliverables will include a **basic UI with a couple of working actions**, which can be used to prove the concept and refine the integration. Subsequent iterations will add more endpoints and handle more complex scenarios (like error recovery, idempotency of requests, etc.). The final goal is a robust UI service ready to be used in the production deployment pipeline as well as interactively by developers.

This HTML UI service, in summary, will provide a crucial interface for deploying cloud infrastructure easily and securely. It marries the power of infrastructure-as-code automation (similar in spirit to Terraform/CloudFormation) with the convenience of a custom-tailored UI and the flexibility of an existing build orchestrator. Once completed, it will significantly streamline how environments are created and managed, allowing the team to deploy to AWS on demand with confidence and minimal manual overhead.
